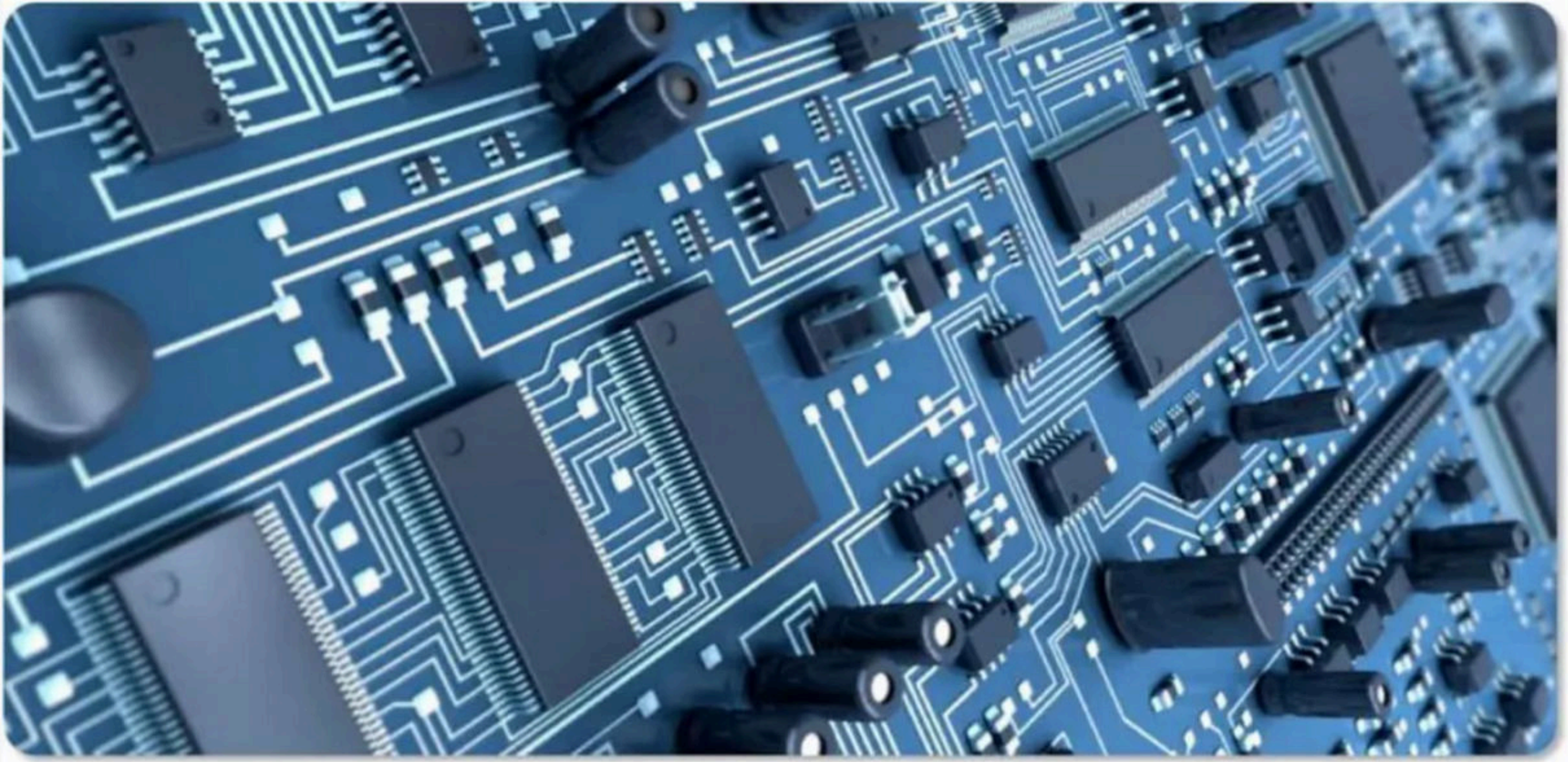
# Boolean Algebra & Expression Part - II

Foundation Course on Digital Electronics

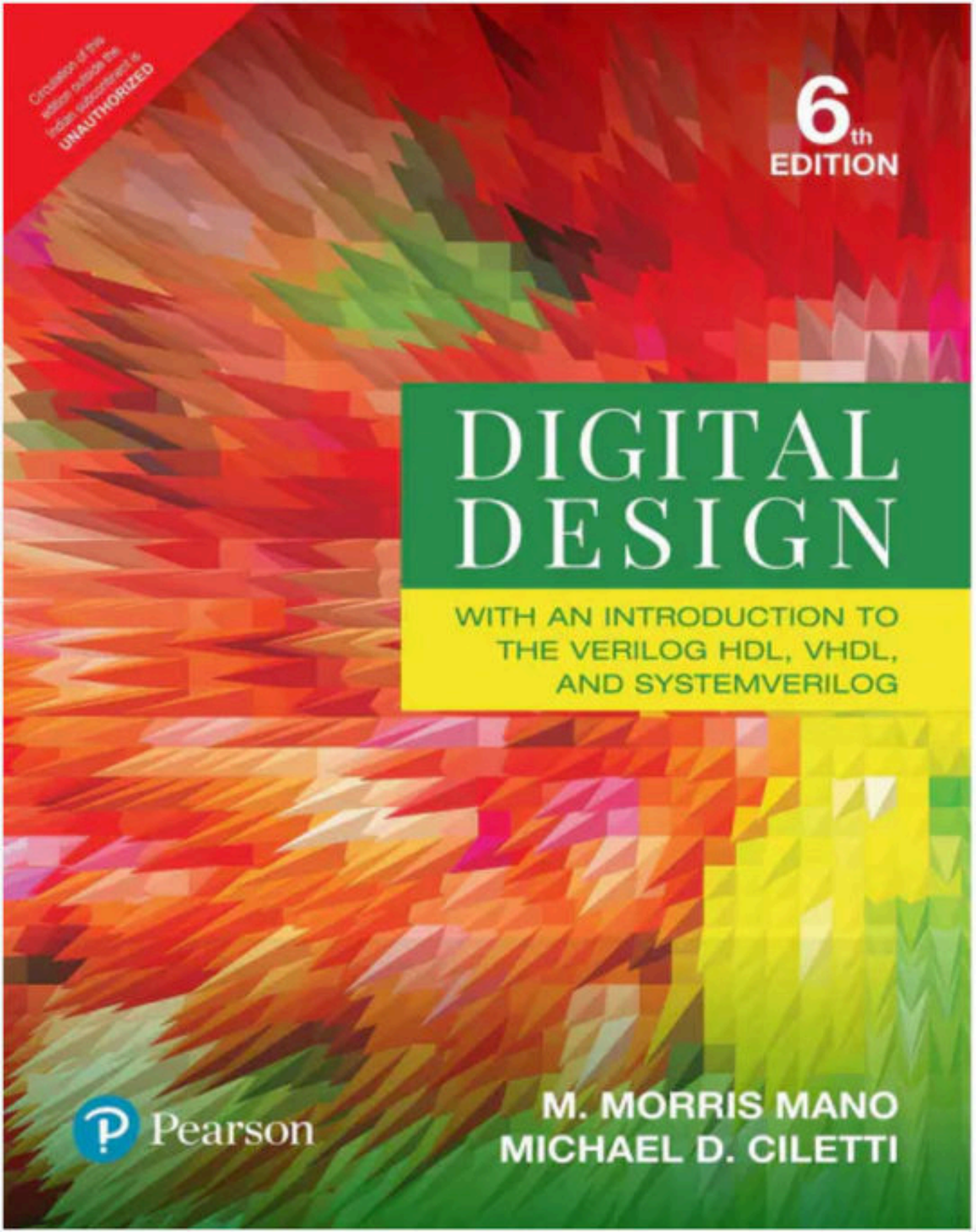
**Sanchit Jain** • Lesson 2 • May 8, 2021

# Digital Electronics

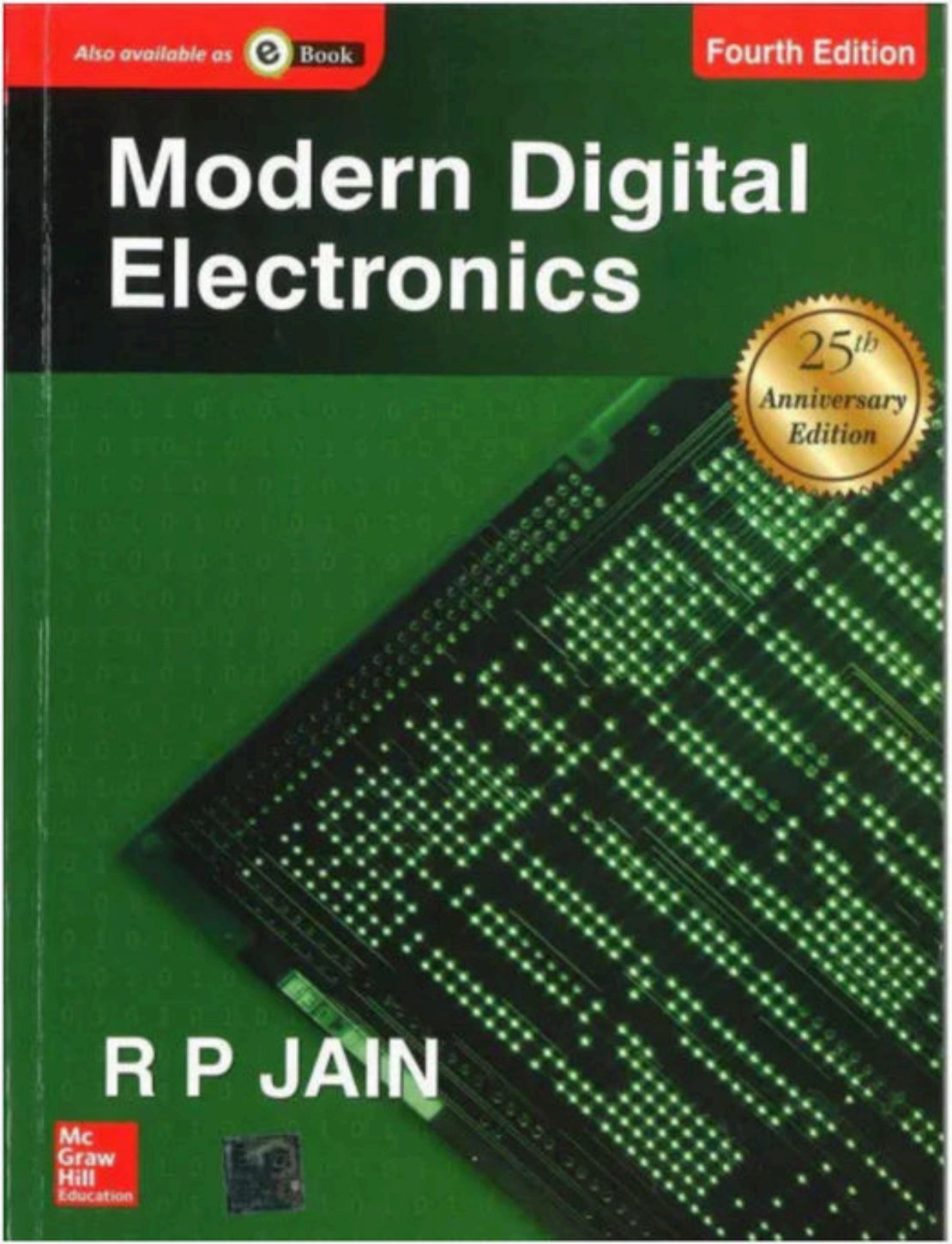


# Digital Electronics

- Core subjects for CS/IT Students
- In GATE 7-8 Marks out of 100 Marks, and 5-6 questions on an average
- In NET 20-22 Marks out of 200 marks and 10-12 questions
- Most questions are Numerical
- Needs less time, good scoring
- Not asked in Industry




- **Book Name** - Digital Design
- **Writers** -
  - M. Morris Mano
  - Michel D. Ciletti
- **Publisher** – Pearson
- **Edition** – 6th




- Book Name – Modern Digital Electronics
- Writers – R P Jain
- Publisher – McGraw Hill
- Edition – 4th

# Syllabus

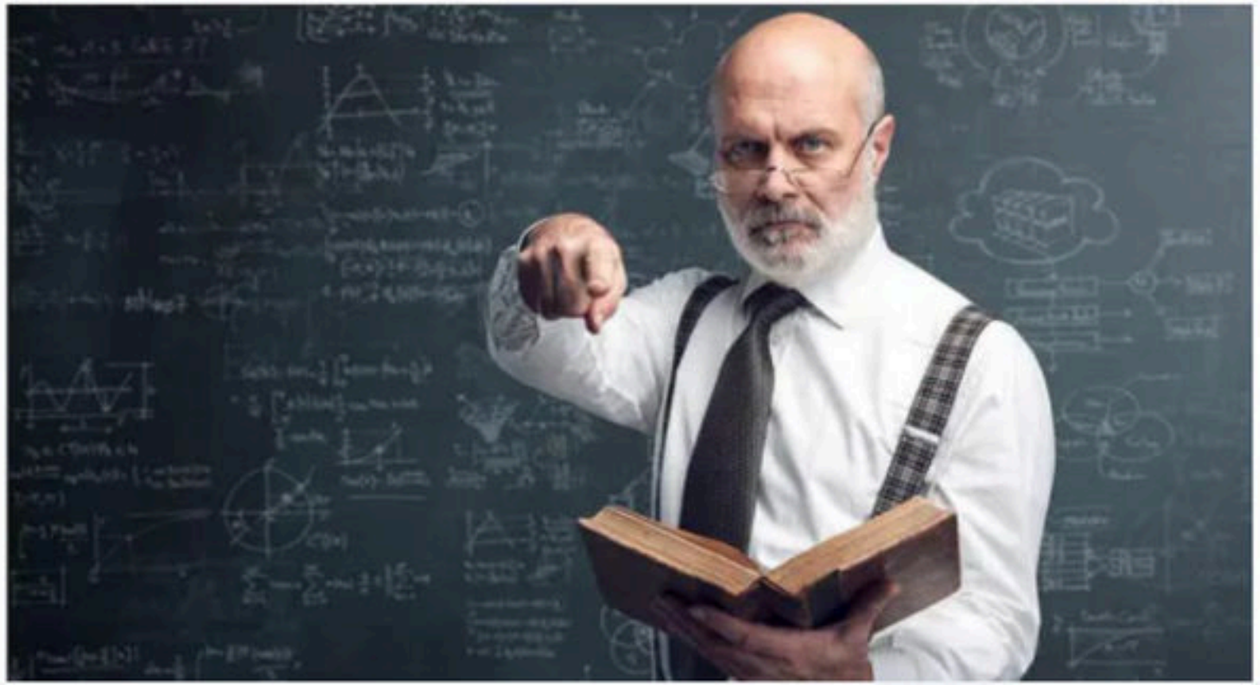
- Understanding Digital Electronics (History and Motive)
- Boolean Algebra Laws and Logic Gates
- Boolean Expression (SOP and POS) (Minimization)
- Combinational Circuit
- Sequential Circuit
- Number System and Number Representation

# What you can expect from me

- Will take care of theory and numerical both
- Will give more weightage to the topics that are asked more frequently in GATE
- Will not emphasize more than required, on a topic
- Will provide PDF of related books

# What i expect from you

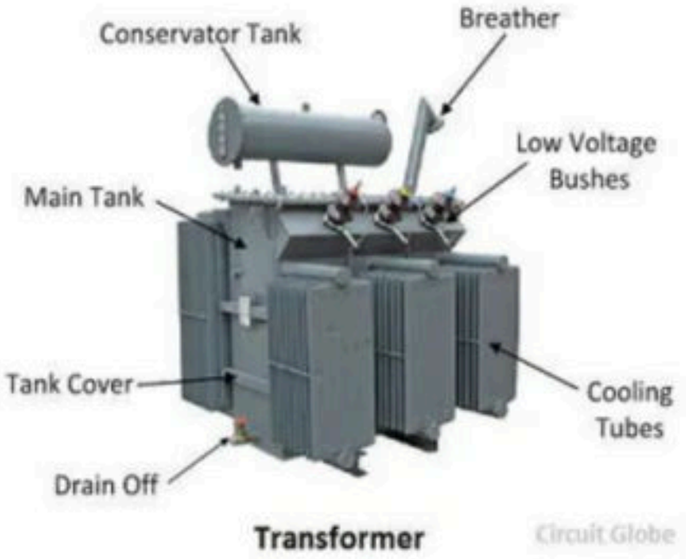
- There is no hurry, feel free to ask questions any time through out the class, but first listen
- Please revise the entire lecture before and after the class
- Be regular, Consistency is most important
- More you practice, more clarity you will get
- If we follow all the above specified points Success in guaranteed



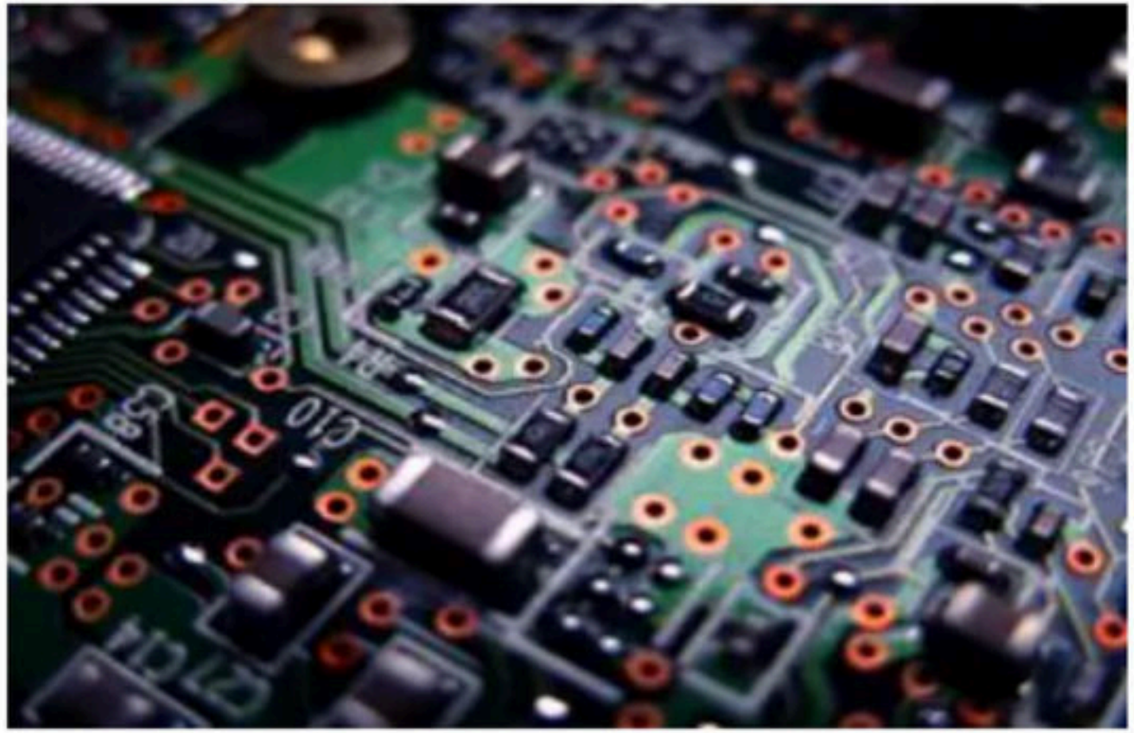
# Break

# Basics

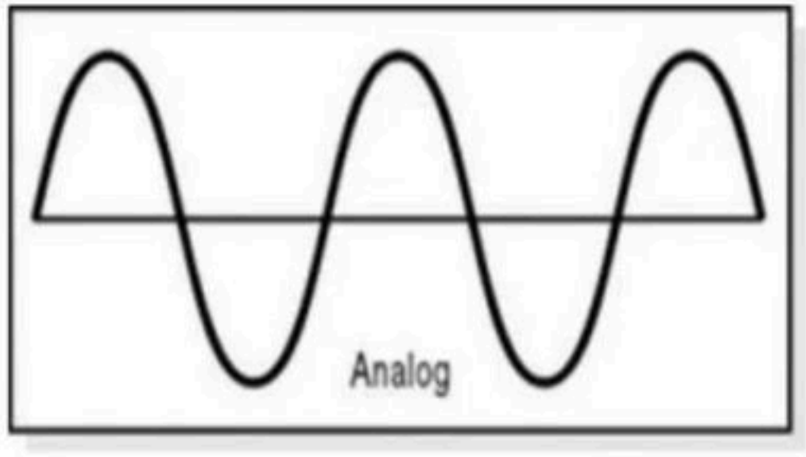
- **Electrical engineering** is a professional engineering discipline that generally deals with the study and application of electricity, electronics and electromagnetism and heavy voltage devices like transformers, motors etc.

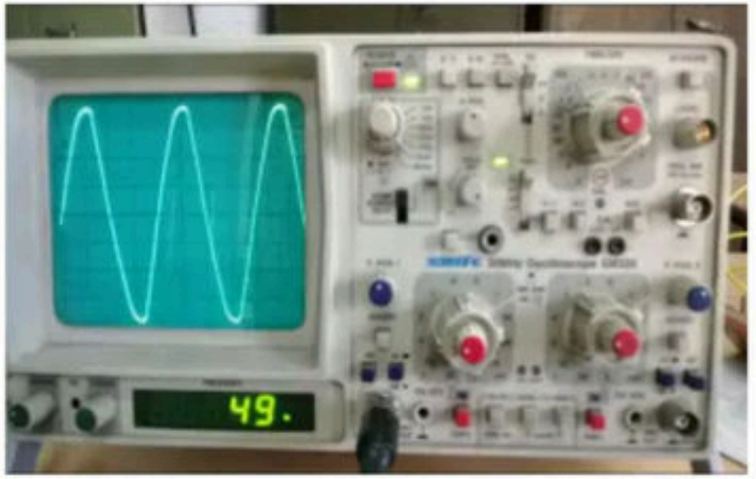



- **Electronic engineering** is an electrical engineering discipline where we work on low voltage devices (such as semiconductor devices, especially transistor, diodes and integrated circuits) to design electronic circuits, VLSI devices and systems.

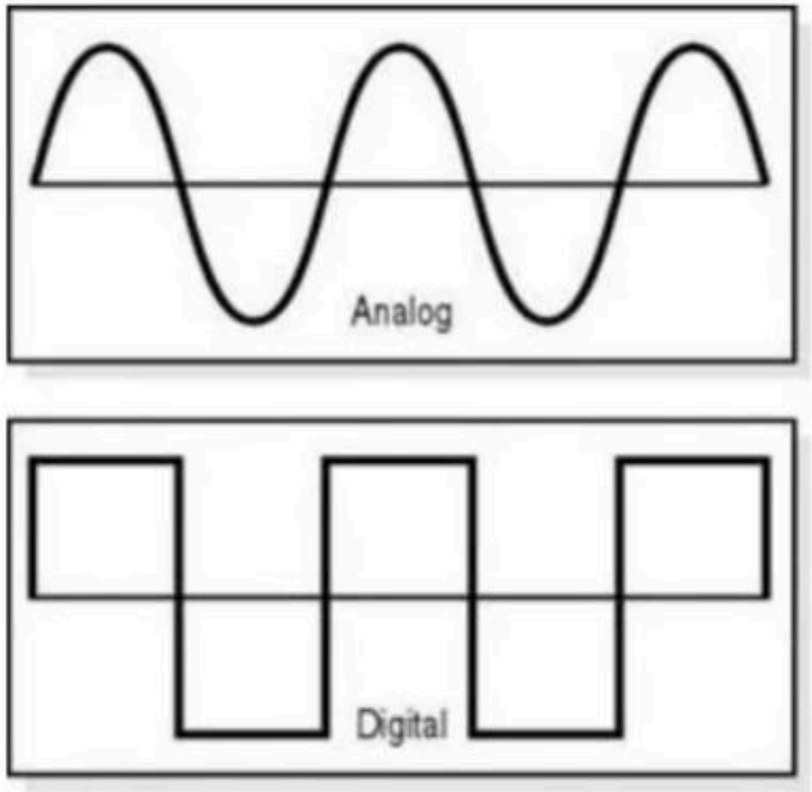


- Electronic systems are generally of two types –
  - **Analog system** in an analog representation, a continuous value is used to denote the information. Analog signal is defined as any physical quantity which varies continuously with respect to time. e.g. amplifier, cro, ecg. Watch, radio.



- **Digital system** – the information is denoted by a finite sequence of discrete value or digits. Digital signal is defined as any physical quantity having discrete values. E.g. digital watch, calculator, bp machine, thermometer etc.





# Advantage of Digital system

**1)** Digital system are easy to design because they do not require sound engineering and mathematical knowledge, uses only switching circuit.

**2)** Storage for long time and processing of information is easy.

**3)** They provide better accuracy and precision compared to analog devices, as digital devices are less affected by noise, sound, electric or magnetic field etc.

**4)** Better modularity and easy fabrication. It means the of digital devices are very small compared to analog devices. E.g. integrated circuits.

**5)** Less cost.

# Disadvantage of Digital system

- Only analog signal is available in the real world, so an extra hardware is required to convert this analog signal to digital signal by using analog to digital converter.

# Conclusion

- Digital systems have such a prominent role in everyday life that we refer to the present technological period as digital age.

- Digital system are used in communication, business, transactions, traffic control, space guidance, medical treatment, weather forecasting, the internet, and many other commercial, industrial and scientific enterprises.

# Note

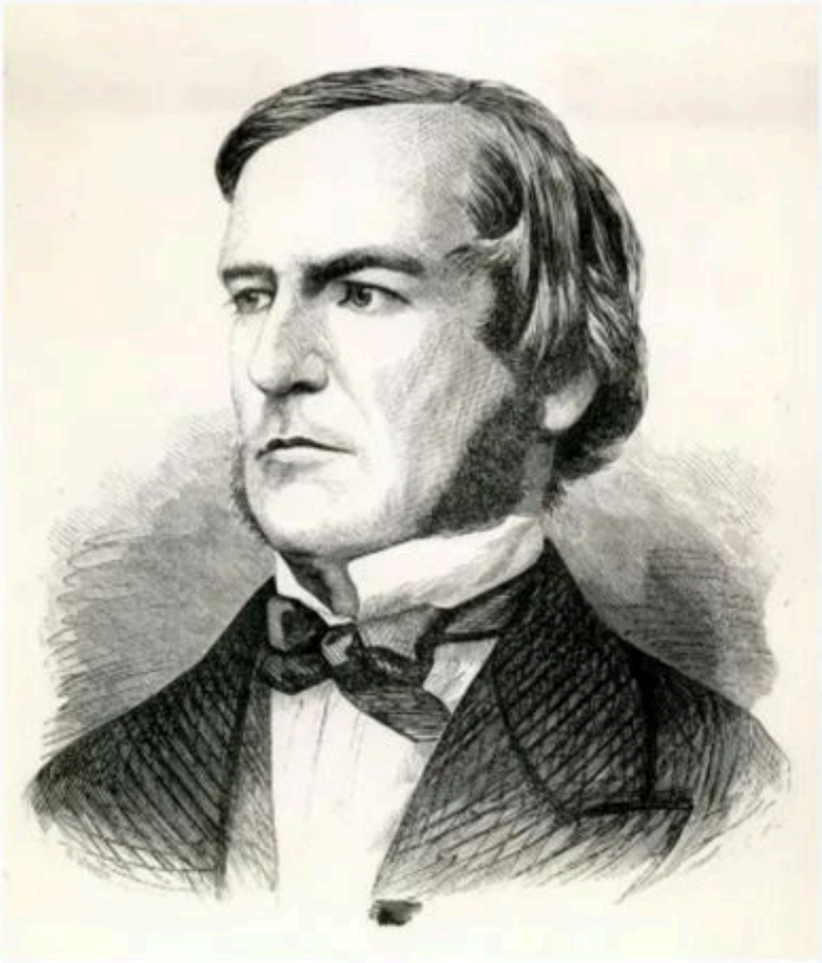
- Early digital computers were used for numeric computation. The discrete elements were the digits, from this application, the term digital computer emerged.

# Break

# Boolean algebra

- The signal in most present day electronic digital system uses just two discrete values and are therefore said to be binary.

- Boolean algebra was introduced by George Boole in his first book The Mathematical Analysis of Logic (1847), and set forth more fully in his An Investigation of the Laws of Thought (1854).



- George boole introduced the concept of binary number system in the studies of the mathematical theory of logic and developed its algebra known as Boolean algebra.

# Boolean algebra

1. In mathematics and mathematical logic, Boolean algebra is the branch of algebra in which the values of the variables are the truth values true and false, usually denoted 1 and 0 respectively.

2. Instead of elementary algebra where the values of the variables are numbers, and the prime operations are addition and multiplication. The main operations of Boolean algebra are
   - The conjunction and denoted as ∧
   - The disjunction or denoted as ∨
   - The negation not denoted as ¬

# Turing Machine

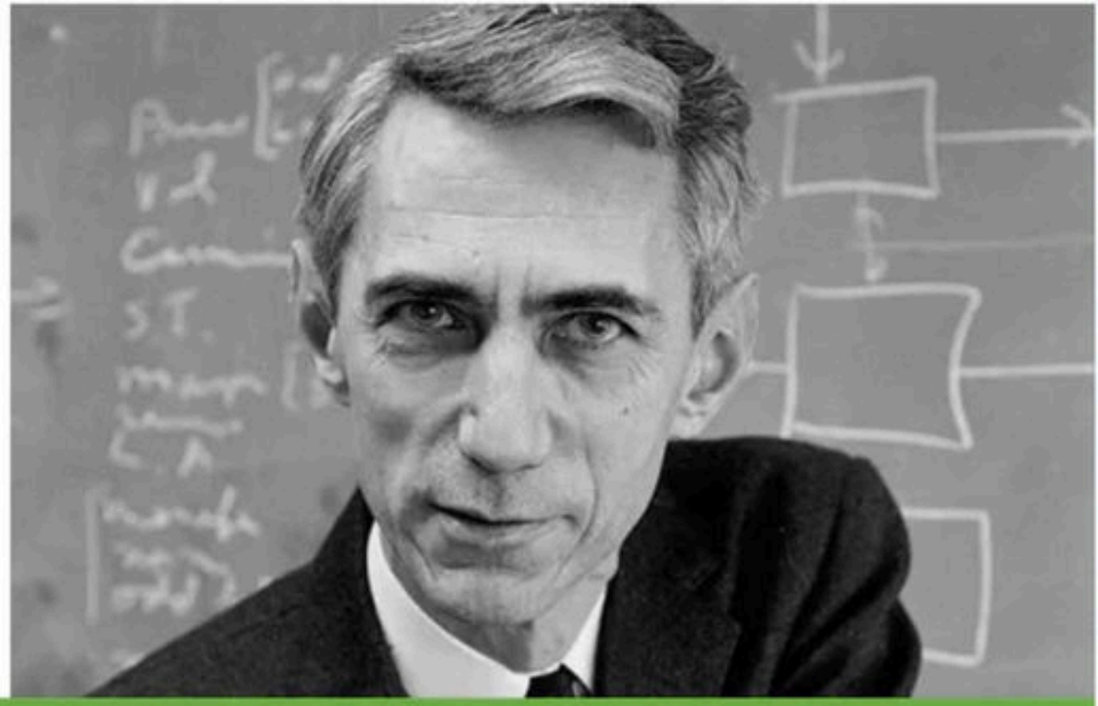
- *The Church-Turing thesis states that any algorithmic procedure that can be carried out by human beings/computer can be carried out by a Turing machine.(1936)*

- It has been universally accepted by computer scientists that the Turing machine provides an ideal theoretical model of a computer.

# Digital System

- In the 1930s, while studying switching circuits, Claude Shannon observed that one could also apply the rules of Boole's algebra in this setting, and he introduced switching algebra as a way to analyze and design circuits by algebraic means in terms of logic gates.

- These logic concepts have been adapted for the design of digital hardware since 1938 Claude Shannon (father of information theory), organized and systematized Boole’s work.



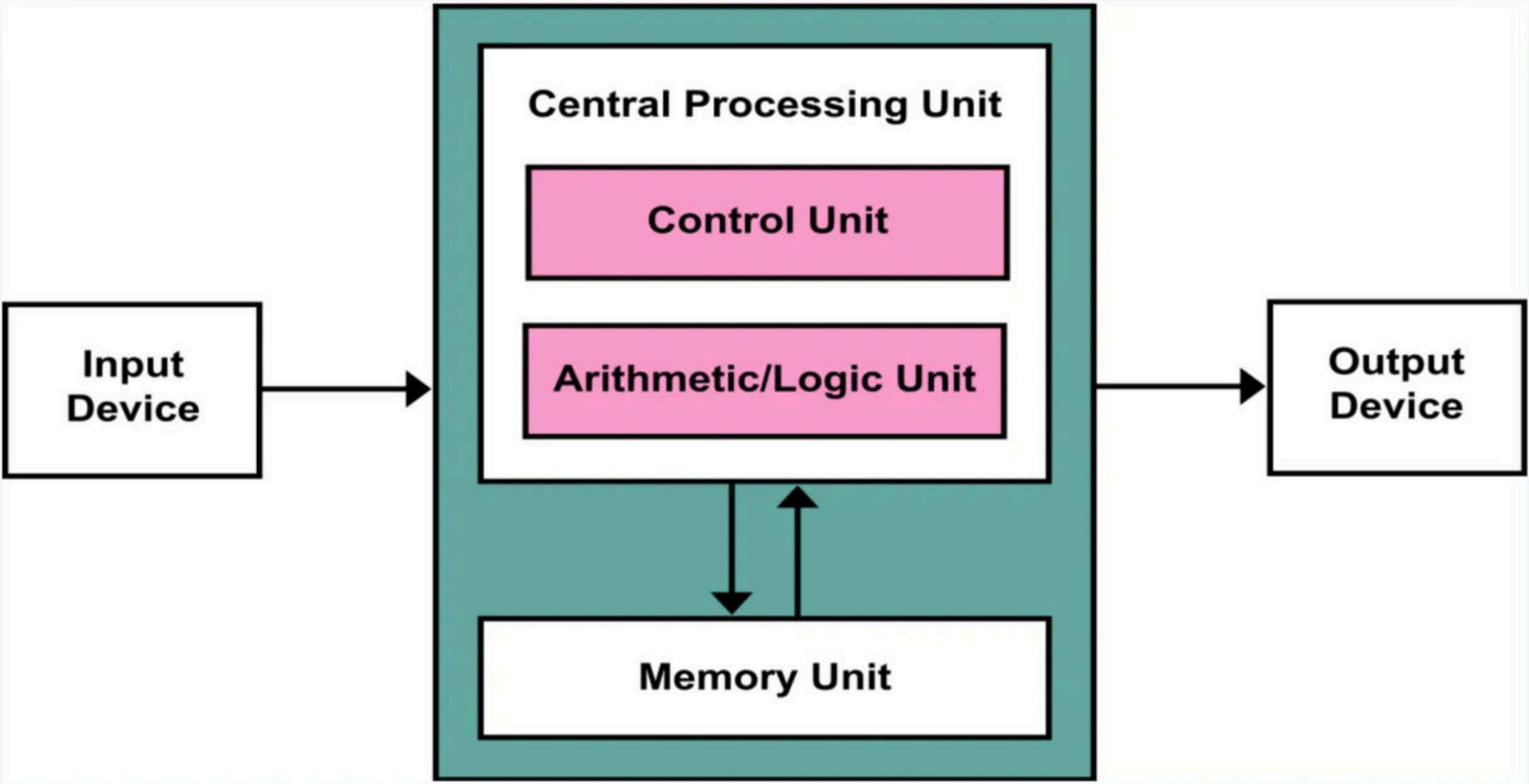
1) Shannon already had at his disposal the Boolean algebra, thus he cast his switching algebra as the two-element Boolean algebra. Efficient implementation of Boolean functions is a fundamental problem in the design of combinational logic circuits.

2) Boolean constants are denoted by 0 or 1. Boolean variables are quantities that can take different values at different times. They may represent input, output or intermediate signals.

3) Here we can have n number of variables usually written as a, b, c…. (lower case) and it satisfy all Boolean laws, which will be discussed later.

# von Neumann architecture

- The **von Neumann architecture** is a computer architecture based on a 1945 description by John von Neumann. That document describes a design architecture for an electronic digital computer with these components:
  - A processing unit that contains an arithmetic logic unit and processor registers
  - A control unit that contains an instruction register and program counter
  - Memory that stores data and instructions
  - External mass storage
  - Input and output mechanisms



# Break

- Digital systems have two logic levels
  - The lower voltage level is called a logic low or logic 0 (0-1 v), which represent digit 0.
  - The higher voltage level is called a logic high or logic 1 (3.5-5 v), which represent digit 1.

# Digital system designing

- Let's try an example of designing a digital device, using this example, we will understand step by step process to design a digital system starting from problem statement.

**Q** Design a digital system for a car manufacturing company, where we want to design a warning signal for a car, there are three inputs, lights of the car(L), day or night(D), ignition (on/off).?

Solution

1) Understand the problem- here we will Understand the definition of the problem

Design the truth table –

| Light | Day | Engine | Warning |
|---|---|---|---|
| 0 | 0 | 0 | |
| 0 | 0 | 1 | |
| 0 | 1 | 0 | |
| 0 | 1 | 1 | |
| 1 | 0 | 0 | |
| 1 | 0 | 1 | |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | |

2) Write the Boolean expression

- $W(L, D, E) = \sum_m (1, 4, 6, 7)$
- $W(L, D, E) = \prod_M (0, 2, 3, 5)$

| Light | Day | Engine | Warning |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

3) Minimize Boolean expression

W = a'b'c + ab'c' + abc' + abc

| | ab | a'b' | a'b | ab | ab' |
|---|---|---|---|---|---|
| c | | 00 | 01 | 11 | 10 |
| c' | 0 | 0 | 2 | 6 | 4 |
| c | 1 | 1 | 3 | 7 | 5 |

W =

| Light | Day | Engine | Warning |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

4) Implement the expression using logic gates, draw the implementation using logic gates

$W = ac' + ab + a'b'c$

| Light | Day | Engine | Warning |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

# Break

# Boolean Algebra Laws

## Idempotent Law

$a \cdot a = a$

$a + a = a$

## Associative law

$a + (b + c) = (a + b) + c$

$(a \cdot b) \cdot c = a \cdot (b \cdot c)$

## Commutative law

$a + b = b + a$

$a \cdot b = b \cdot a$

## Distributive law

$a + (b \cdot c) = (a + b) \cdot (a + c)$

$a \cdot (b + c) = a \cdot b + a \cdot c$

| a | b | c | a + b | b + c | (a+b)+c | a+(b+c) |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |

# Boolean Algebra Laws

**Idempotent Law**

$a \cdot a = a$

$a + a = a$

**Associative law**

$a.(b.c) = (a.b).c$

$a + (b + c) = (a + b) + c$

**Commutative law**

$a.b = b.a$

$a + b = b + a$

**Distributive law**

$a.(b + c) = a.b + a.c$

$a + (b.c) = (a + b).(a + c)$

**De-Morgan law**

$$\overline{(a+b)} = \bar{a} \cdot \bar{b}$$

$$\overline{(a \cdot b)} = \bar{a} + \bar{b}$$

$+ \rightarrow \cup$

$\cdot \rightarrow \cap$

$1 \rightarrow U$

$0 \rightarrow \phi$

**Identity law**

1. $a + 0 = a$
2. $a \cdot 1 = a$
3. $a + 1 = 1$
4. $a \cdot 0 = 0$

$a \cup \phi = a$

$a \cap U = a$

$a \cup U = U$

$a \cap \phi = \phi$

**Complementation law**

1. $\bar{0} = 1$
2. $\bar{T} = 0$
3. $a + \bar{a} = 1$
4. $a \cdot \bar{a} = 0$

$A \cup \bar{A} = U$

$A \cap \bar{A} = \phi$

| $a$ | $1$ | $a+1$ |
|---|---|---|
| 0 | 1 | 1 |
| 1 | 1 | 1 |

**Involution law**

$$\overline{\bar{a}} = a$$

**De-Morgan law**

$(a + b)' = a' \cdot b'$

$(a. b)' = a' + b'$

**Identity law**

$a + 0 = a$

$a. 0 = 0$

$a + 1 = 1$

$a. 1 = a$

**Complementation law**

$0' = 1$

$1' = 0$

$a. a' = 0$

$a + a' = 1$

**Involution law**

$(a')' = a$

# Break

**Q** The idempotent law in Boolean algebra says that: **(NET-JUNE-2008)**

**(A)** ~(~x) = x

**(B)** x + x = x

**(C)** x + xy = x

**(D)** x(x + y) = x

**Q** A V A = A is called: **(NET-DEC-2004)**

**(A)** Identity law

**(B)** De Morgan's law

+ = V

. = ∧

**(C)** Idempotent law

-VE /MS Q

**(D)** Complement law

45

1 marks

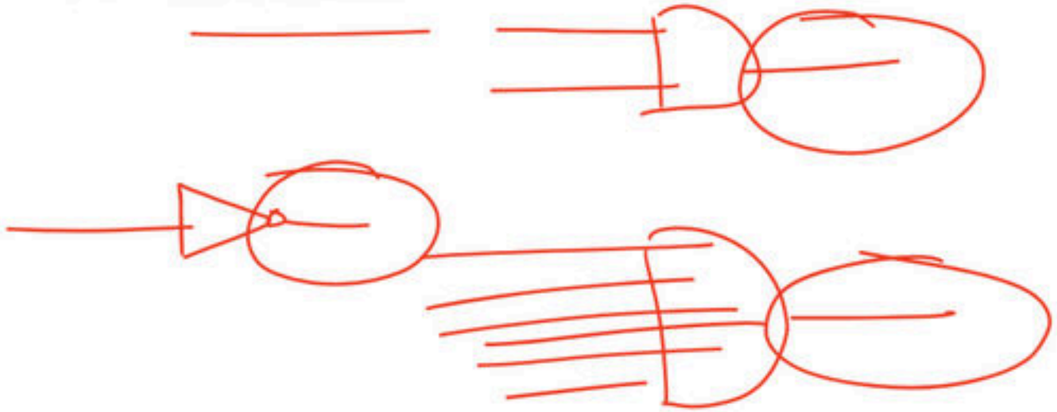
30 sec

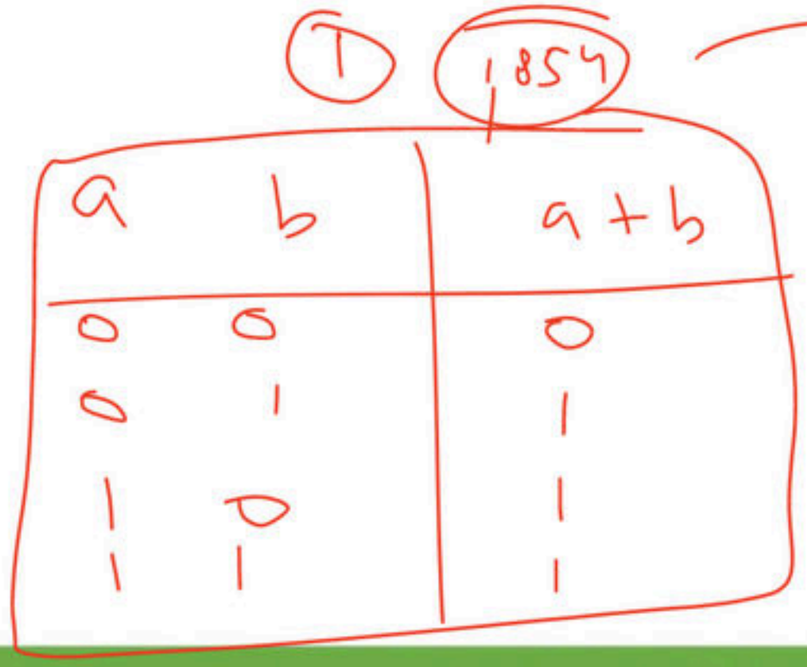
2 marks

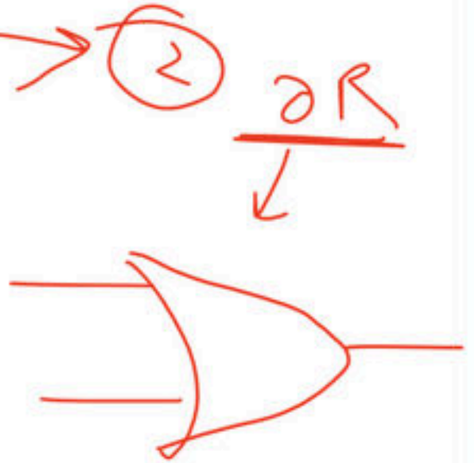
Bora

# Logic gate

- In electronics, a logic gate is a physical device implementing a Boolean function
- Logic gate performs a logical operation on one or more binary inputs signals and produces a single binary output signal.
- Logic gate is the basic building block from which many kinds of logic circuits can be constructed.

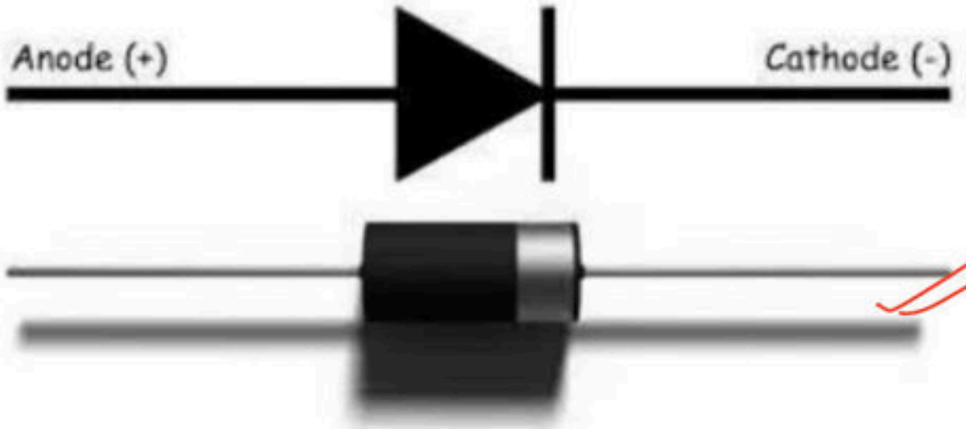


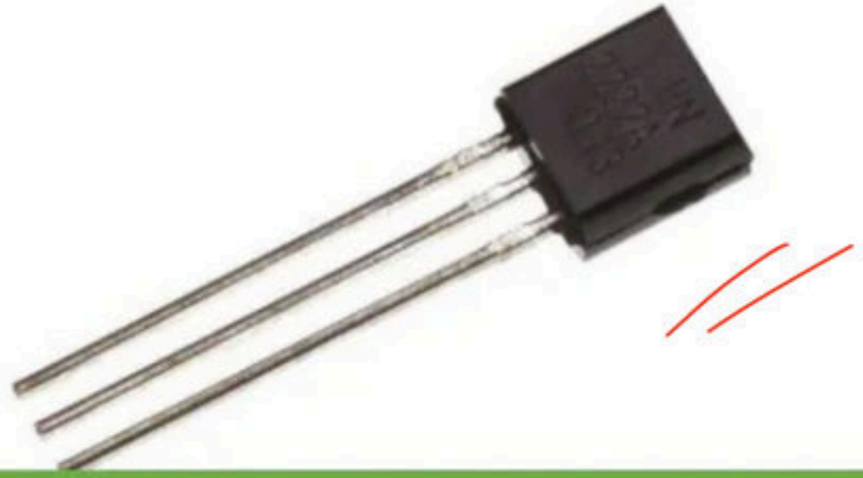


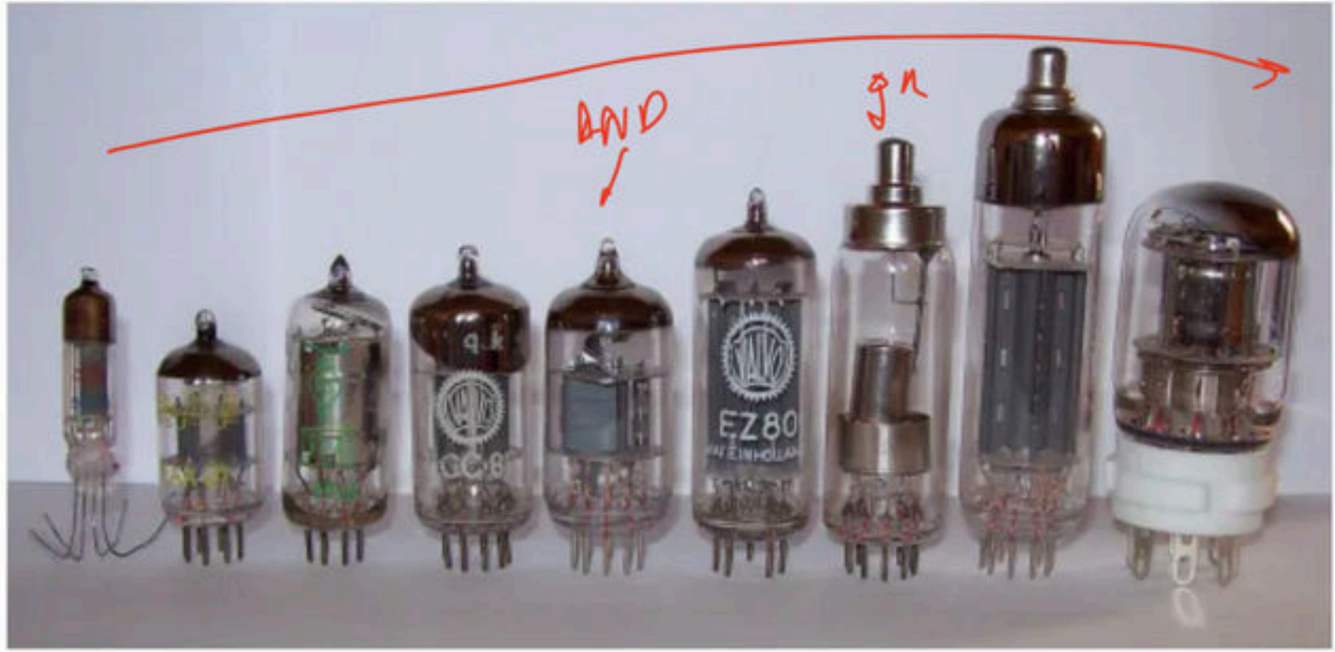



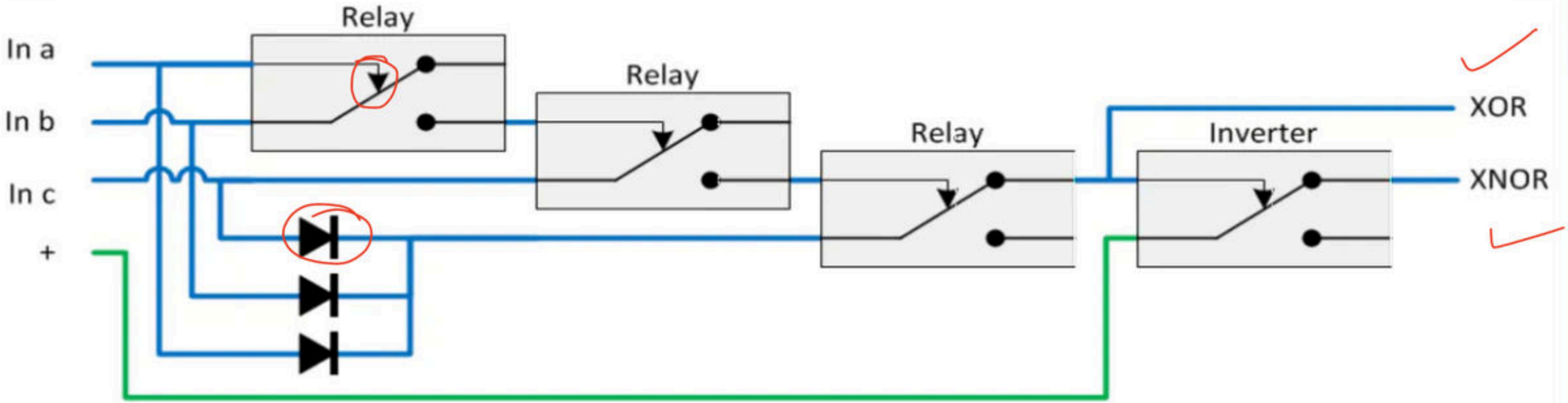
# Logic gate

- Logic gates are primarily implemented using diodes or transistors acting as electronic switches, but can also be constructed using vacuum tubes, electromagnetic relays (relay logic), fluidic logic, pneumatic logic, optics, molecules, or even mechanical elements.

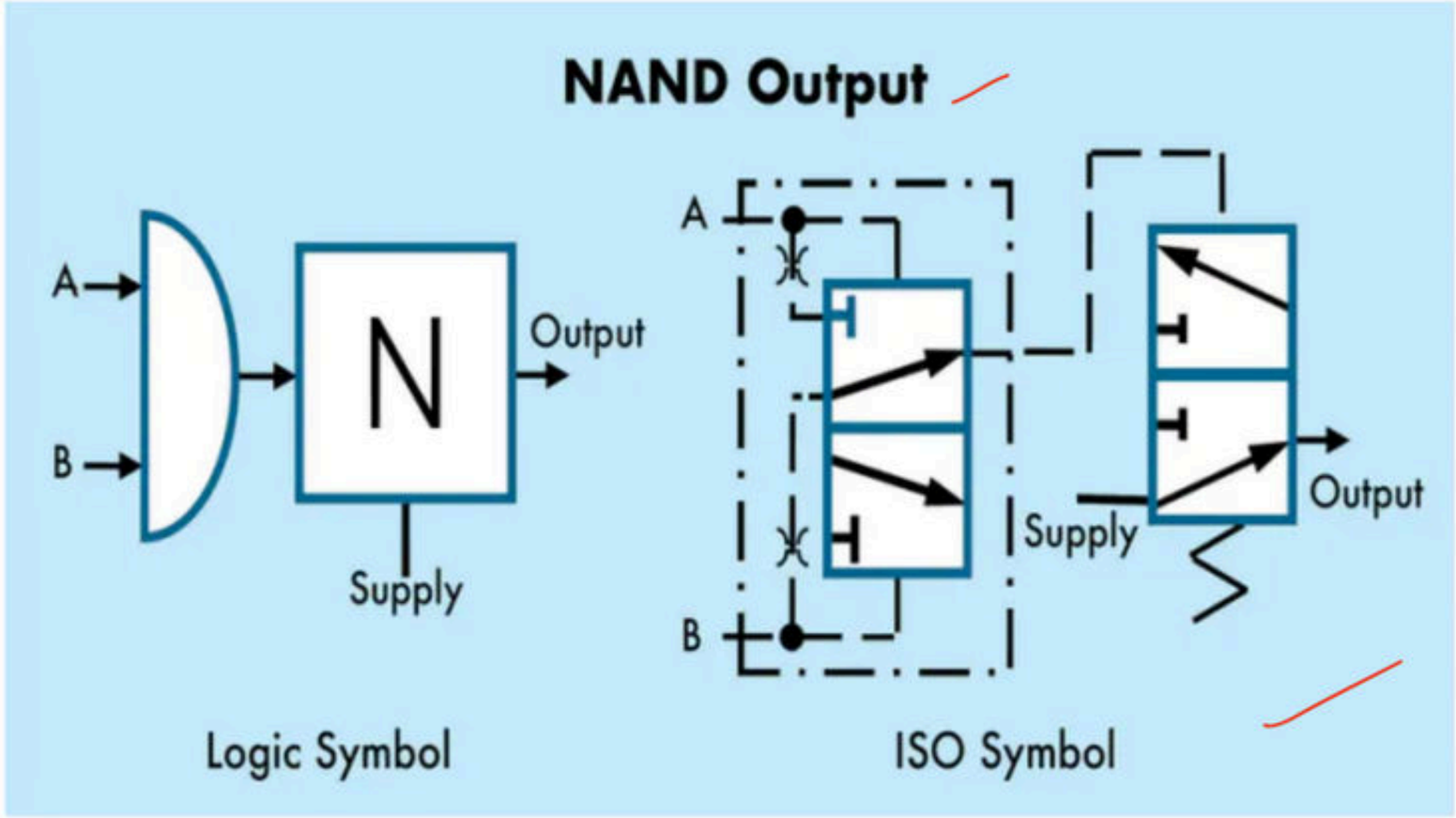







electromagnetic relays

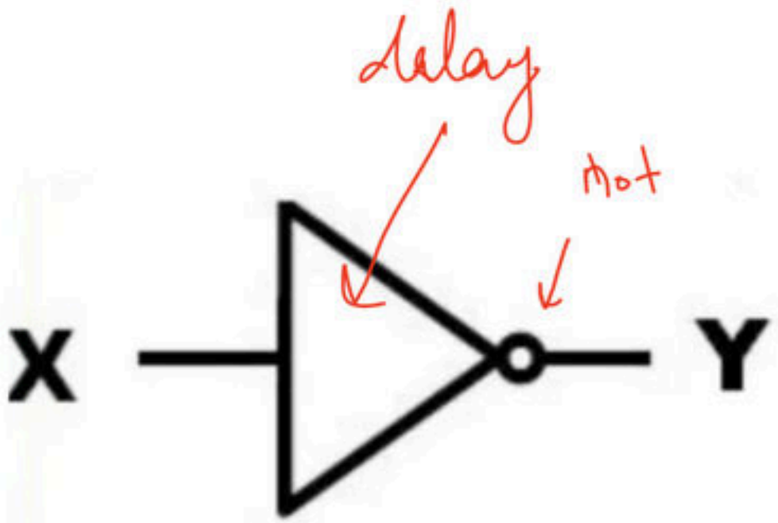



pneumatic logic

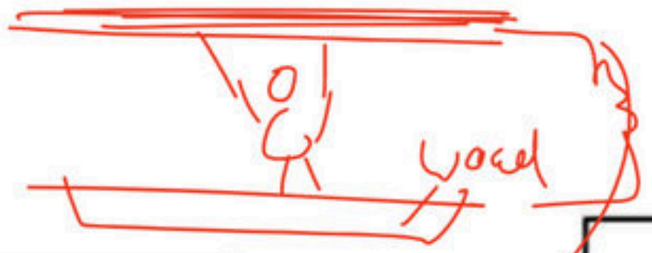
# Break

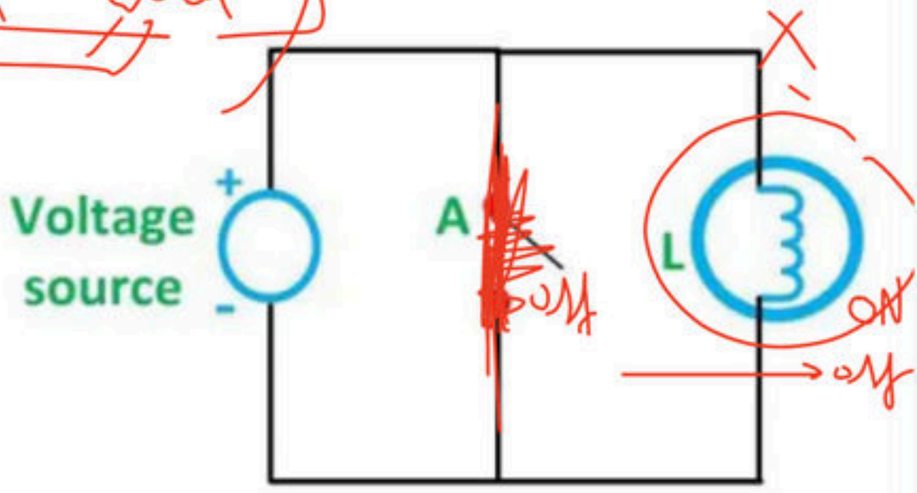
# Not Gate(Inverter)

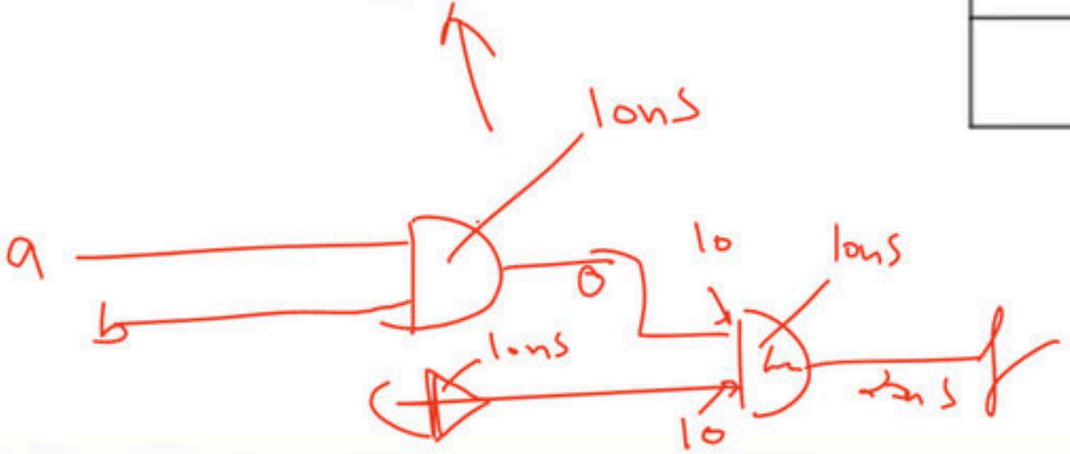
- It represents not logical operator is also known as inverter, it is a unary operator, which simply complement the input.

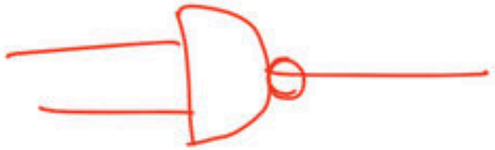


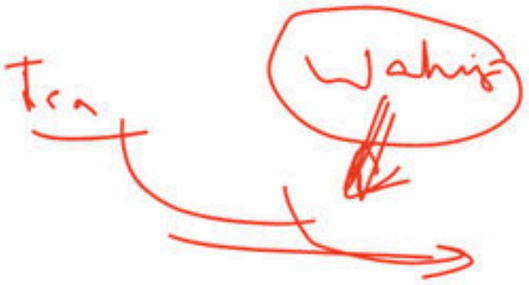

| Truth Table | |
|---|---|
| Input | Output |
| X | Y = X' |
| 0 | 1 |
| 1 | 0 |





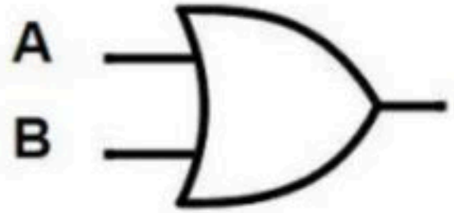




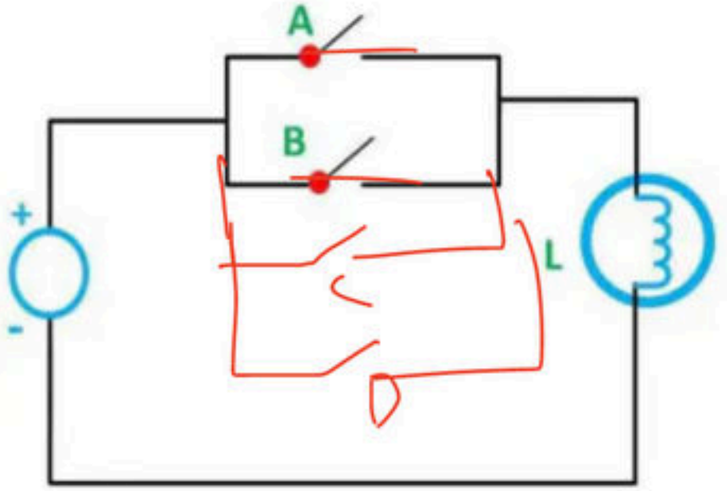
# Break

# OR Gate

- It is a digital logic gate, that implements logical disjunction. The output will be high if at least one of the input lines is high.




| Truth Table | | |
|---|---|---|
| Input | | Output |
| A | B | Y = A + B |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

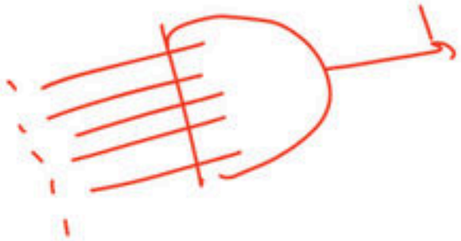


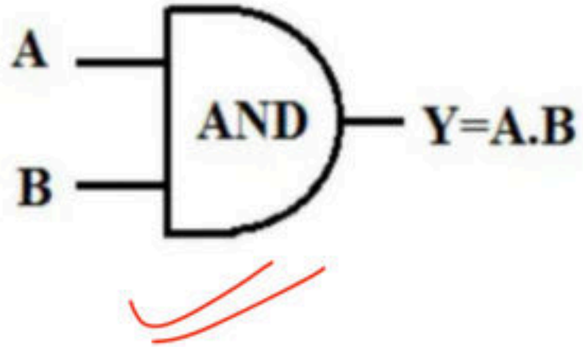
- OR gate satisfy all the 3 rules idempotent, associative, and commutative.
  1. **Idempotent Law:** $a + a = a$
  2. **Associative law:** $a + (b + c) = (a + b) + c$
  3. **Commutative law:** $a + b = b + a$

# Break

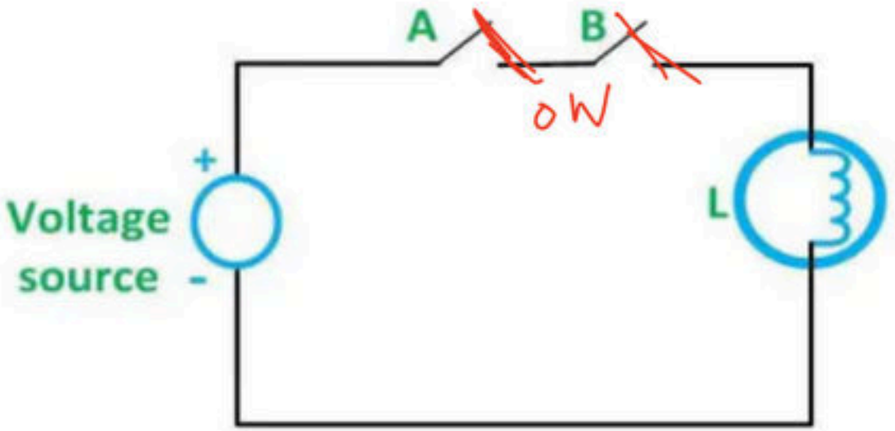
# And Gate

- It is a digital logic gate, that implements logical conjunction. Output will be high if and only if all input are high otherwise low.






| Truth Table | | |
|---|---|---|
| Input | | Output |
| A | B | Y = A . B |
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

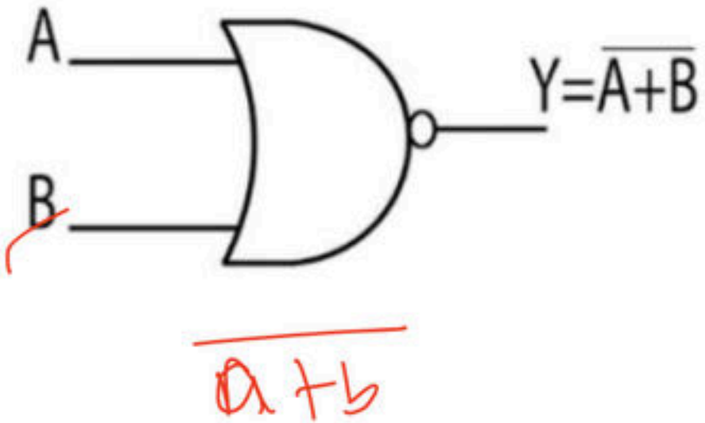


- And gate satisfy all the 3 rules idempotent, associative, and commutative.
  1. **Idempotent Law:** a. a = a
  2. **Associative law:** a.(b. c) = (a. b). C
  3. **Commutative law:** a. b = b. a

# Break

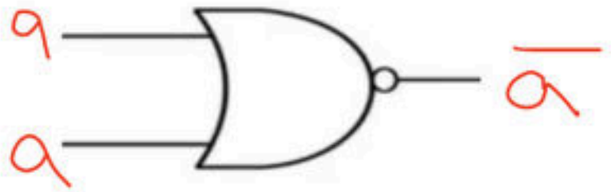
# Nor gate

- The output will be high if and only if all inputs are low. Or simply a OR gate followed by an invertor.
- NOR gate is also called universal gate because it can be used to implement any other logic gate. We will cover this property extensively in the next chapter.

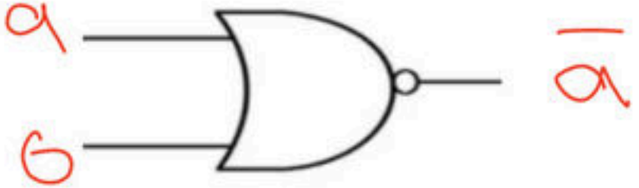



| Truth Table | | |
|---|---|---|
| Input | | Output |
| A | B | Y = (A + B)' |
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 0 |

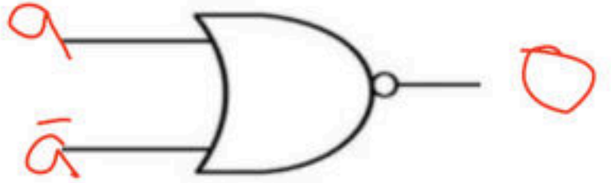
1. NOR with same gives comp
   - (a + a)' = (a)' =



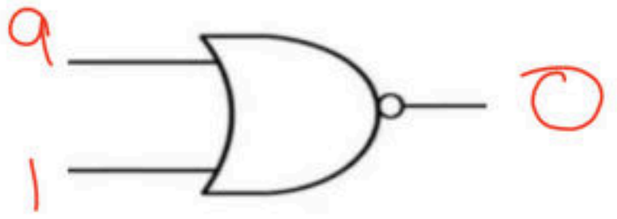

2. NOR with zero gives comp
   - (a + 0)' = a'




3. NOR with complement gives zero
   - (a + a')' = 1' = 0




4. NOR with one gives zero
   - (a + 1)' = 1' = 0



- Idempotent and associative law
  - $(a + a)' \neq a$
  - $((a + b)' + c)' \neq (a + (b + c)')'$
- Commutative law.
  - $(a + b)' = (b + a)'$

$$\overline{(\overline{(a+b)}+c)}$$

$$(a+b)\cdot\bar{c}$$

$$(a+b)\cdot\bar{c}$$

$$a\bar{c}+b\bar{c}$$

$$\neq$$

$$\overline{a+\overline{(b+c)}}$$

$$\bar{a}\cdot\overline{(b+c)}$$

$$\bar{a}\cdot(b+c)$$

$$\bar{a}b+\bar{a}c$$

L.H.S = R.H.S

# Conclusion

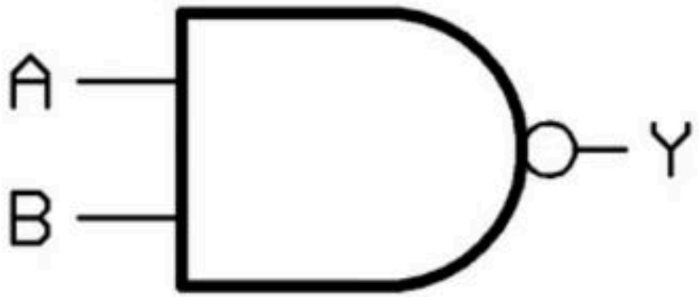
- NOR with same gives complement →(a + a)’ = a’
- NOR with zero gives complement → (a + 0)’ = a’
- NOR with complement gives zero → (a + a’)’ = 0
- NOR with one gives zero → (a + 1)’ = 0

- NOR does not satisfy idempotent and associative law
  - (a + a)’ != a
  - ((a + b)’+ c)’ != (a + (b + c)’)’
- It satisfies commutative law.
  - (a + b)’ = (b + a)’

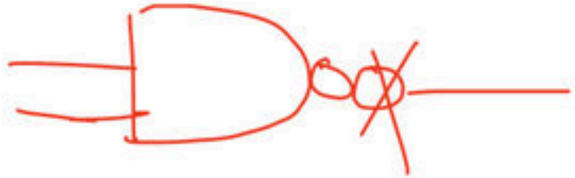
# Break

# NAND Gate

- The output will be low if and only if all inputs are high. Or simply an and gate followed by an invertor
- NAND gate is also called universal gate because it can be used to implement any other logic gate. We will cover this property extensively in the next chapter.




| Truth Table | | |
|---|---|---|
| Input | | Output |
| A | B | Y = (A . B)' |
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

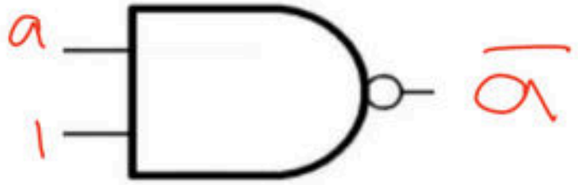


1. NAND with ZERO give one
   - (a. 0)' = $\overline{0} = 1$


2. NAND with COMPLEMENT give one
   - (a. a')' = $\overline{0} = 1$


3. NAND with SAME give comp
   - (a. a)' = $\overline{a}$


4. NAND with ONE give comp
   - (a. 1)' = $\overline{a}$



- NAND with idempotent and associative law
  - (a. a)’ ≠ a
  - ((a. b)’. c)’ ≠ (a. (b. c)’)’

- NAND with commutative law
  - (a. b)’ = (b. a)’

# Conclusion

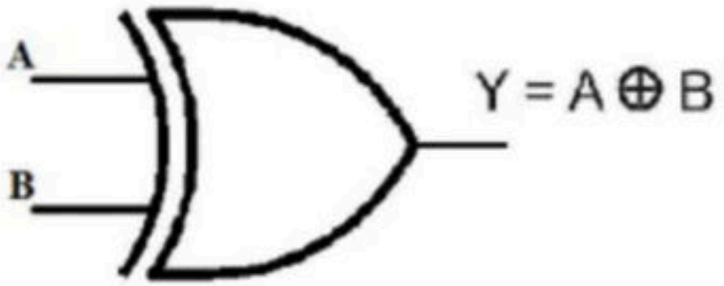
- NAND with ZERO give ONE → (a. 0)’ = 1
- NAND with COMPLEMENT give ONE → (a. a’)’ = 1
- NAND with SAME give COMPLEMENT → (a. 1)’ = a’
- NAND with ONE give COMPLEMENT → (a. a)’ = a’

- NAND does not satisfy idempotent and associative law
  - (a. a)’ != a
  - ((a. b)’. c)’ != (a. (b. c)’)’
- It satisfies commutative law
  - (a. b)’ = (b. a)’

# Break

# EX-OR

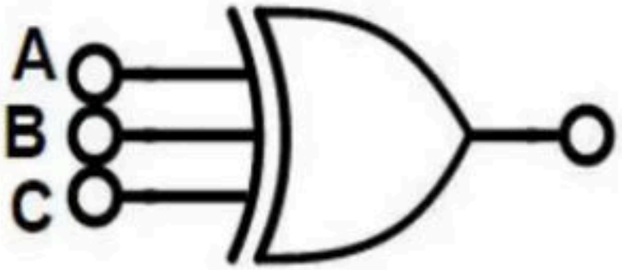
- For two inputs, output will be high if and only if both the input values are different.
- $a \oplus b = a'.b + a.b'$




| Truth Table | | |
|---|---|---|
| Input | | Output |
| A | B | Y = A ⊕ B |
| 0 | 0 | |
| 0 | 1 | |
| 1 | 0 | |
| 1 | 1 | |

# EX-OR

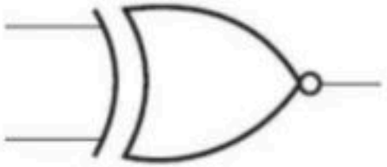
- The XOR gate is a digital logic gate that gives High as output when the number of inputs High are odd.



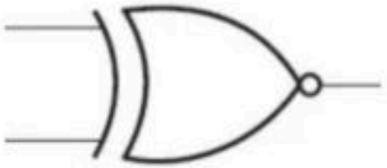

| A | B | C | a ⊕ b ⊕ c |
|---|---|---|---|
| 0 | 0 | 0 | |
| 0 | 0 | 1 | |
| 0 | 1 | 0 | |
| 0 | 1 | 1 | |
| 1 | 0 | 0 | |
| 1 | 0 | 1 | |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | |

1. EX-OR with ZERO give_____
   - (a $\oplus$ 0) =

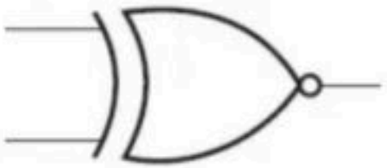
2. EX-OR with ONE give_____
   - (a $\oplus$ 1) =



3. EX-OR with SAME give_____
   - (a $\oplus$ a) =



4. EX-OR with COMPLEMENT give_____
   - (a $\oplus$ a') =



- EX-OR with idempotent law
  - $(a \oplus a)$ ☐ $a$

- EX-OR with associative and commutative law.
  - $((a \oplus b) \oplus c)$ ☐ $(a \oplus (b \oplus c))$
  - $(a \oplus b)$ ☐ $(b \oplus a)$

# Conclusion

- EX-OR with ZERO give SAME → $(a \oplus 0) = a$
- EX-OR with ONE give COMPLEMENT → $(a \oplus 1) = a'$
- EX-OR with SAME give ZERO → $(a \oplus a) = 0$
- EX-OR with COMPLEMENT give ONE → $(a \oplus a') = 1$

- EX-OR does not satisfy idempotent
  - $(a \oplus a) != a$
- It satisfies associative and commutative law.
  - $((a \oplus b) \oplus c) = (a \oplus (b \oplus c))$
  - $(a \oplus b) = (b \oplus a)$

# Break

**Q** Which of the following logic expressions is incorrect? **(NET-JULY-2016)**

**a)** $1 \oplus 0 = 1$

**b)** $1 \oplus 1 \oplus 1 = 1$

**c)** $1 \oplus 1 \oplus 0 = 1$

**d)** $1 \oplus 1 = 0$

**Q** Consider the Boolean operator with the following properties: **(GATE-2016) (1 Marks)**

x # 0 = x,

x # 1 = x’,

x # x = 0

and x # x’ = 1

Then x # y is equivalent to

**a)** xy’ + x’y **b)** xy’ + x’y’ **c)** x’y + xy **d)** xy + x’y’

**Q** The binary operator # is defined by the following truth table. **(GATE-2015) (1 Marks)**

Which of the following is true about the binary operator # ?

**a)** Both commutative and associative

**b)** Commutative but not associative

**c)** Not commutative but associative

**d)** Neither commutative nor associative

| p | q | P # q |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

**Q** Let $\oplus$ denote the Exclusive OR (XOR) operation. Let '1' and '0' denote the binary constants. Consider the following Boolean expression for F over two variables P and Q.

$F(P, Q) = ( ( 1 \oplus P) \oplus (P \oplus Q) ) \oplus ( (P \oplus Q) \oplus (Q \oplus 0) )$

The equivalent expression for F is

**(GATE-2014) (2 Marks)**

**(A)** P + Q

**(B)** (P + Q)'

**(C)** $P \oplus Q$

**(D)** $(P \oplus Q)'$

**Q** If $A \oplus B = C$, then which of the following are invalid: **(NET-JUNE-2007)**

**(A)** $A \oplus C = B$ **(B)** $B \oplus C = A$ **(C)** $A \oplus B \oplus C = 1$ **(D)** $A \oplus B \oplus C = 0$

| A | B | $A \oplus B = C$ |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

**Q** Consider the following sequence of instructions: **(NET-DEC-2005)**

$a = a \oplus b$, $b = a \oplus b$, $a = b \oplus a$. This sequence

**(A)** retains the value of the a and b

**(B)** complements the value of a and b

**(C)** swap a and b

**(D)** negates values of a and b

**Q** An example of a connective which is not associative is: **(NET-DEC-2004)**

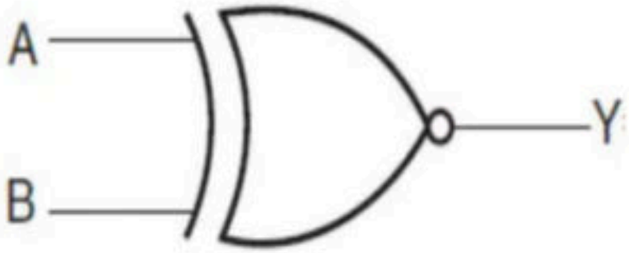
**(A)** AND **(B)** OR **(C)** EX-OR **(D)** NAND

Q A ⊕ P ⊕ A ⊕ P ⊕ B ⊕ P ⊕ B ⊕ P ⊕ B?

# Break

# EX-NOR

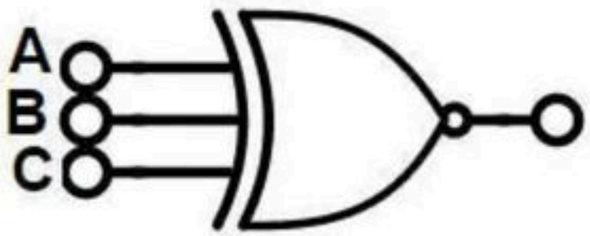
- For two input, output will be high if and only if both the input values are same
- $a \odot b = a'.b' + a.b$




| Truth Table | | |
|---|---|---|
| Input | | Output |
| A | B | Y = A ⊙ B |
| 0 | 0 | |
| 0 | 1 | |
| 1 | 0 | |
| 1 | 1 | |

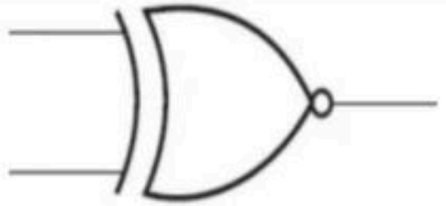
# EX-NOR

- The EX-NOR gate is a digital logic gate that gives output High when the number inputs low are even.

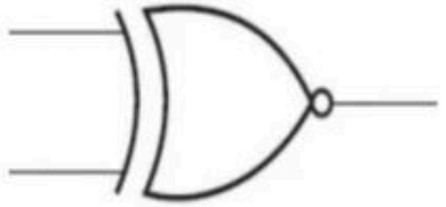



| A | B | C | a ⊙ b ⊙ c |
|---|---|---|---|
| 0 | 0 | 0 | |
| 0 | 0 | 1 | |
| 0 | 1 | 0 | |
| 0 | 1 | 1 | |
| 1 | 0 | 0 | |
| 1 | 0 | 1 | |
| 1 | 1 | 0 | |
| 1 | 1 | 1 | |

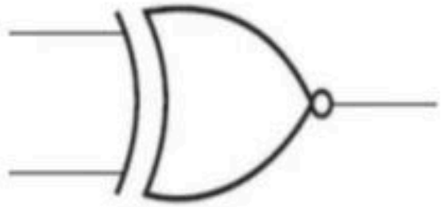
1. EX-NOR with ZERO give_____
   - (a ⊙ 0) =



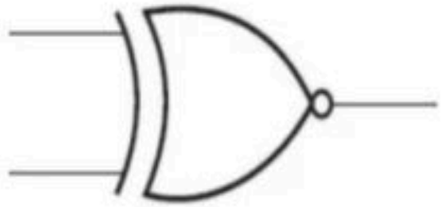
2. EX-NOR with ONE give_____
   - (a ⊙ 1) =



3. EX-NOR with SAME give_____
   - (a ⊙ a) =



4. EX-NOR with COMPLEMENT give_____
   - (a ⊙ a') =



- EX-NOR with idempotent law
  - (a ⊙ a) ☐ a

- EX-NOR with associative and commutative law
  - ((a ⊙ b) ⊙c) ☐ (a ⊙ (b ⊙ c))
  - (a ⊙ b) ☐ (b ⊙ a)

# Conclusion

- EX-NOR with ZERO give COMPLEMENT →(a ⊙ 0) = a'
- EX-NOR with ONE give SAME →(a ⊙ 1) = a
- EX-NOR with SAME give ONE →(a ⊙ a) = 1
- EX-NOR with COMPLEMENT give ZERO →(a ⊙ a') = 0

- EX-NOR does not satisfy idempotent
  - (a ⊙ a) != a
- it satisfies associative and commutative law.
  - ((a ⊙ b) ⊙c) = (a ⊙ (b ⊙ c))
  - (a ⊙ b) = (b ⊙ a)

# Break

**Q** A Boolean operator s is defined as follows:

1 s 1 = 1, 1 s 0 = 0, 0 s 1 = 0 and 0 s 0 = 1

What will be the truth value of the expression (x s y) s z = x s (y s z)? **(NET-DEC-2013)**

**(A)** Always false

**(B)** Always true

**(C)** Sometimes true

**(D)** True when x, y, z are all true

**Q** Define the connective * for the Boolean variables X and Y as: X * Y = XY + X' Y'. Let Z = X * Y.

**(GATE-2007) (2 Marks)**

Consider the following expressions P, Q and R.

P: X = Y⋆Z  Q: Y = X⋆Z  R: X⋆Y⋆Z=1

| X | Y | X ⊙ Y = Z |
|---|---|---|
| | | |
| | | |
| | | |
| | | |

Which of the following is TRUE?

**(A)** Only P and Q are valid

**(B)** Only Q and R are valid.

**(C)** Only P and R are valid.

**(D)** All P, Q, R are valid.

# Break

| A | B | C | $a \oplus b \oplus c$ | $a \odot b \odot c$ |
|---|---|---|---|---|
| 0 | 0 | 0 | | |
| 0 | 0 | 1 | | |
| 0 | 1 | 0 | | |
| 0 | 1 | 1 | | |
| 1 | 0 | 0 | | |
| 1 | 0 | 1 | | |
| 1 | 1 | 0 | | |
| 1 | 1 | 1 | | |

# Conclusion

| A | B | C | $a \oplus b \oplus c$ | $a \odot b \odot c$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

- Ex-or and ex-nor gate behaves as a complement of each other if the number of input variable is even.
- Ex-or and ex-nor gate behave same if no of input variables are odd.

# Conclusion

1. Ex-or and ex-nor gate behaves as a complement of each other if the number of input variable is even.

2. Ex-or and ex-nor gate behave same if no of input variables are odd.

# Relations or EX-OR and EX-NOR

$a \oplus b = a'\square b = a\square b' = (a\square b)' = (a'\square b')' = a'\square b' = (a'\square b)' = (a\square b')'$

$a \odot b = a'\square b = a\square b' = (a\square b)' = (a'\square b')' = a'\square b' = (a'\square b)' = (a\square b')'$

# Conclusion

$a \oplus b = a' \odot b = a \odot b' = (a \odot b)' = (a' \odot b')' = a' \oplus b' = (a' \oplus b)' = (a \oplus b')'$

$a \odot b = a' \oplus b = a \oplus b' = (a \oplus b)' = (a' \oplus b')' = a' \odot b' = (a' \odot b)' = (a \odot b')'$

**Q** Which one of the following is NOT a valid identity? **(GATE-2019) (2 Marks)**

**(A)** $(x \oplus y) \oplus z = x \oplus (y \oplus z)$

**(B)** $(x + y) \oplus z = x \oplus (y + z)$

**(C)** $x \oplus y = x + y$, if $xy = 0$

**(D)** $x \oplus y = (xy + x'y')'$

**Q** Let $\oplus$ and $\odot$ denote the Exclusive OR and Exclusive NOR operations, respectively.

Which one of the following is NOT CORRECT? **(GATE-2018) (2 Marks)**

**(A)** $(P \oplus Q)' = P \odot Q$

**(B)** $\bar{P} \oplus Q = P \odot Q$

**(C)** $\bar{P} \oplus \bar{Q} = P \oplus Q$

**(D)** $(P \oplus \bar{P}) \oplus Q = (P \odot \bar{P}) \odot \bar{Q}$

**Q** Let, $X_1 \oplus X_2 \oplus X_3 \oplus X_4 = 0$ where $X_1$, $X_2$, $X_3$, $X_4$ are Boolean variables, and $\oplus$ is the XOR operator. Which one of the following must always be TRUE? **(GATE-2016)**

**(2 Marks)**

**a)** $X_1X_2X_3X_4 = 0$

**b)** $X_1X_3+X_2 = 0$

**c)** $X'_1 \oplus X'_3 = X'_2 \oplus X'_4$

**d)** $X_1 + X_2 + X_3 + X_4 = 0$

| | $X_1$ | $X_2$ | $X_3$ | $X_4$ | $X_1 \oplus X_2 \oplus X_3 \oplus X_4 = 0$ | $X_1X_2X_3X_4 = 0$ | $X_1X_3+X_2 = 0$ | $X'_1 \oplus X'_3 = X'_2 \oplus X'_4$ | $X_1 + X_2 + X_3 + X_4 = 0$ |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | | | | |
| 1 | 0 | 0 | 0 | 1 | | | | | |
| 2 | 0 | 0 | 1 | 0 | | | | | |
| 3 | 0 | 0 | 1 | 1 | | | | | |
| 4 | 0 | 1 | 0 | 0 | | | | | |
| 5 | 0 | 1 | 0 | 1 | | | | | |
| 6 | 0 | 1 | 1 | 0 | | | | | |
| 7 | 0 | 1 | 1 | 1 | | | | | |
| 8 | 1 | 0 | 0 | 0 | | | | | |
| 9 | 1 | 0 | 0 | 1 | | | | | |
| 10 | 1 | 0 | 1 | 0 | | | | | |
| 11 | 1 | 0 | 1 | 1 | | | | | |
| 12 | 1 | 1 | 0 | 0 | | | | | |
| 13 | 1 | 1 | 0 | 1 | | | | | |
| 14 | 1 | 1 | 1 | 0 | | | | | |
| 15 | 1 | 1 | 1 | 1 | | | | | |

**Q** Which one of the following expressions does NOT represent exclusive NOR of x and y? **(GATE-2013) (1 Marks)**

**(A)** xy + x' y'

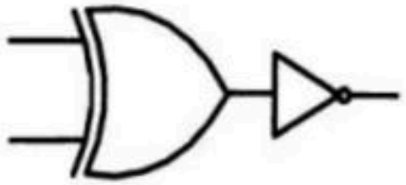
**(B)** x ^ y' where ^ is XOR

**(C)** x' ^ y where ^ is XOR

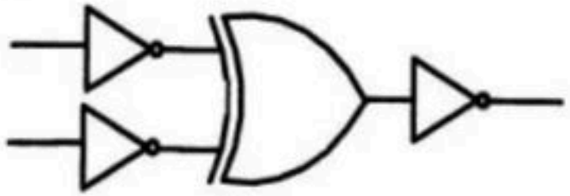
**(D)** x' ^ y' where ^ is XOR

**Q** Which one of the following circuits is NOT equivalent to a 2-input XNOR (exclusive NOR) gate? **(GATE-2011) (1 Marks)**

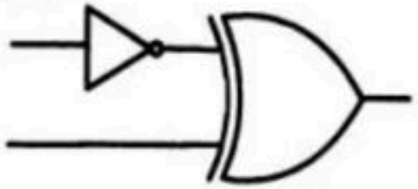
(A)



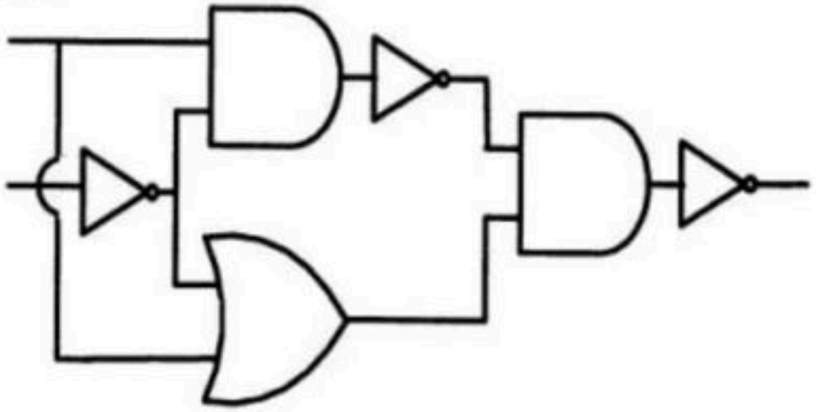
(B)



(C)



(D)



**Q** The simultaneous equations on the Boolean variables x, y, z and w,

$x + y + z = 1$

$xy = 0$

$xz + w = 1$

$xy + z'w' = 0$

have the following solution for x, y, z and w, respectively. **(GATE-2000) (2 Marks)**

**(A)** 0 1 0 0

**(B)** 1 1 0 1

**(C)** 1 0 1 1

**(D)** 1 0 0 0

# Months Wise Calendar for GATE-2022 (Unacademy Plus) (Sanchit Jain)

| Month | | Morning Class (7:00am – 9:00am) | Evening Class (7:00pm – 9:00pm) |
|---|---|---|---|
| Feb-2021 | 1 | DS & Programming (15-Feb to 23-Mar) (56 Hours) | DM (22-Feb to 27-Mar) (50 Hours) |
| Mar-2021 | | Algo (24-Mar to 26-April) (38 Hours) | DBMS (29-Mar to 30-April) (50 Hours) |
| Apr-2021 | | CN(29-April to 3-June) (54 Hours) | DE (5-May to 4-June) (40 Hours) |
| May-2021 | | TOC (7-June to 26-July) (58 Hours) | COA (7-June to 5-July) (38 Hours) |
| Jun-2021 | | Compiler (27-July to 24-Aug) (34 Hours) | OS (6-July to 18-Aug) (58 Hours) |
| Jul-2021 | | | |
| Aug-2021 | | | |
| Sep-2021 | 2 | DM<br>DBMS<br>DE<br>COA<br>OS | DS & Programming<br>Algo<br>CN<br>TOC<br>Compiler |
| Oct-2021 | | | |
| Nov-2021 | | | |
| Dec-2021 | | | |
| Jan-2022 | | | |
| Feb-2022 | | | |

# Boolean Expressions

- Boolean expressions are the method using which we save the information about the Boolean function, that when we get value 1 and when we get value 0 as output.

- So, we convert the truth table of the function into an expression, reading which we can understand where the output is 1 and when it is zero.

| Light | Day | Engine | Warning |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

- W = a’b’c + ab’c’ + abc’ + abc

- There are two popular approaches for writing these expressions.
  - Sum of Product (SOP) (which remember when we get 1)
  - Product of Sum (POS) (which remember when we get 0)

- Make a note of this as we are studying Boolean function remembering both 0 and 1 is not required, so we can either concentrate on 0 or 1.

| Light | Day | Engine | Warning |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

- $W(L, D, E) = \sum_m (1, 4, 6, 7)$
- $W(L, D, E) = \prod_M (0, 2, 3, 5)$

- Power of SOP and POS bother are same, i.e. any Boolean functions can be represented using both SOP and POS.

- Mathematically speaking we should choose (0 or 1), whatever is less in number because then it will be easy to represent

| A | B | $f_1$ = | $f_2$ = |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 |

**Q** The truth table represents the Boolean function **(GATE-2012) (1 Marks)**

**(A)** X **(B)** X+Y **(C)** $X \oplus Y$ **(D)** Y

| X | Y | F (X, Y) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

# Break

# SOP (sum of product)

- A sum of product form expression contains product terms (AND terms) which are sum (OR) together, that's why called sum of product.

- Each product terms (AND terms) consists of one or more literals (variables) appearing either in complements or uncomplemented form. E.g. a'b + b'c' + abc

- A product term which contains all the literals (variables) either in complemented or uncomplemented form is called minterm.

- In a n variable function, there will be $2^n$ minterms.

| Binary Representation | Sequence | Minterm | Designation |
|---|---|---|---|
| 000 | 0 | a'b'c' | $m_0$ |
| 001 | 1 | a'b'c | $m_1$ |
| 010 | 2 | a'bc' | $m_2$ |
| 011 | 3 | a'bc | $m_3$ |
| 100 | 4 | ab'c' | $m_4$ |
| 101 | 5 | ab'c | $m_5$ |
| 110 | 6 | abc' | $m_6$ |
| 111 | 7 | Abc | $m_7$ |

- The result of a product term must always be 1, If a literal is having value 1 then it is ok, but if not, then we complement those which is 0, it to make it 1.

- There is only 1 input sequence of variables for any minterm on which the output is 1, so it represents information.

- Then the sum of all product term(minterm) to from a function, and functions will have value 1, if at least one of the product term(minterm) is 1.

| Binary Representation | Sequence | Minterm | Designation |
|---|---|---|---|
| 000 | 0 | a'b'c' | $m_0$ |
| 001 | 1 | a'b'c | $m_1$ |
| 010 | 2 | a'bc' | $m_2$ |
| 011 | 3 | a'bc | $m_3$ |
| 100 | 4 | ab'c' | $m_4$ |
| 101 | 5 | ab'c | $m_5$ |
| 110 | 6 | abc' | $m_6$ |
| 111 | 7 | Abc | $m_7$ |

# Canonical logic forms

- Either in POS or SOP form it is not essential that all product or sum terms contains all the literals.

- **Canonical SOP form**: - In a sum of product form expression, if each AND term (product term) consists all the literals(variables) appearing either in complements or uncomplemented form. E.g. a'bc + ab'c' + abc. Then the form is said to be canonical SOP.

**Q** The minterm expansion of f(P, Q, R) = PQ + QR' + PR' is **(GATE-2010) (2 Marks)**

**(A)** $m_2 + m_4 + m_6 + m_7$

**(B)** $m_0 + m_1 + m_3 + m_5$

**(C)** $m_0 + m_1 + m_6 + m_7$

**(D)** $m_2 + m_3 + m_4 + m_5$

**Q** Consider a function and find no of minterms F (a, b, c) = a + a'b + abc + bc'?

# Break

# POS (Product of Sum)

- A product of sum (POS) form of expression contains OR (sum) terms which are AND (product) together, that's why called product of sum expression.

- Each OR term (sum term) consists of one or more literals(variables) appearing either in complemented or uncomplemented form. (a' + b). (b' + c'). (a + c)

- A OR (sum) term which contains all the literals(variables) either in complemented or uncomplemented form is called maxterm. In a n variable function, there will be $2^n$ maxterms.

| Binary Representation | Sequence | Maxterm | Designation |
|---|---|---|---|
| 000 | 0 | a + b + c | $M_0$ |
| 001 | 1 | a + b + c' | $M_1$ |
| 010 | 2 | a + b' + c | $M_2$ |
| 011 | 3 | a + b' + c' | $M_3$ |
| 100 | 4 | a' + b + c | $M_4$ |
| 101 | 5 | a' + b + c' | $M_5$ |
| 110 | 6 | a' + b' + c | $M_6$ |
| 111 | 7 | a' + b' + c' | $M_7$ |

- The result of the OR (sum) term must be 0, so If a variable is having value 0 then it is ok, but if not then we complement the variable it to make it 0.

- There is only 1 input sequence for which the output of a Maxterm is 0. Because, it requires all values as zero.

- Then the product of all sum term (maxterm), from a function, and functions will have a value 0, if any of the sum term(maxterm) is 0.

| Binary Representation | Sequence | Maxterm | Designation |
|---|---|---|---|
| 000 | 0 | a + b + c | $M_0$ |
| 001 | 1 | a + b + c' | $M_1$ |
| 010 | 2 | a + b' + c | $M_2$ |
| 011 | 3 | a + b' + c' | $M_3$ |
| 100 | 4 | a' + b + c | $M_4$ |
| 101 | 5 | a' + b + c' | $M_5$ |
| 110 | 6 | a' + b' + c | $M_6$ |
| 111 | 7 | a' + b' + c' | $M_7$ |

# Canonical logic forms

- **Canonical POS form**: - In a product of sum form expression, if each OR term (sum term) consists all the literals(variables) appearing either in complements or uncomplemented form. E.g. (a' + b + c). (a + b' + c'). (a + b + c). Then the form is said to be Canonical POS form.

| Binary Representation | Sequence | Maxterm | Designation |
|---|---|---|---|
| 000 | 0 | a + b + c | $M_0$ |
| 001 | 1 | a + b + c' | $M_1$ |
| 010 | 2 | a + b' + c | $M_2$ |
| 011 | 3 | a + b' + c' | $M_3$ |
| 100 | 4 | a' + b + c | $M_4$ |
| 101 | 5 | a' + b + c' | $M_5$ |
| 110 | 6 | a' + b' + c | $M_6$ |
| 111 | 7 | a' + b' + c' | $M_7$ |

**Q** Given the function F = P'+QR, where F is a function in three Boolean variables P, Q and R and P' =! P, consider the following statements. **(GATE-2015) (2 Marks)**

**S1**: F = Σ (4, 5, 6)

**S2**: F = Σ (0, 1, 2, 3, 7)

**S3**: F = Π (4, 5, 6)

**S4**: F = Π (0, 1, 2, 3, 7)

Which of the following is true?

**(A)** S1-False, S2-True, S3-True, S4-False

**(B)** S1-True, S2-False, S3-False, S4-True

**(C)** S1-False, S2-False, S3-True, S4-True

**(D)** S1-True, S2-True, S3-False, S4-False

# Break

# No of functions possible

**Q** With n-Boolean variables how many different Boolean functions are possible?

# No of functions possible

**Q** With n-Boolean variables how many different Boolean functions are possible?

Solution: with n binary variable we can generate $2^n$ combinations. And as we know that a Boolean function can also have only 2 possible values either 0 or 1, so total number of different functions possible will be $2^{(2^n)}$.

Similarly, we can generalize this idea as $x^{(y^z)}$ are the total number of functions possible, x is the nature of the function, y is the nature of the variable and z is the number of variables.

**Q** How many different Boolean functions of degree 4 are there? **(NET-JUNE-2013)**

**(A)** $2^4$ **(B)** $2^8$ **(C)** $2^{12}$ **(D)** $2^{16}$

**Q** What is the maximum number of different Boolean functions involving n Boolean variables? **(GATE-2007) (1 Marks)**

**a)** $n^2$ **b)** $2^n$ **c)** $2^{2\wedge n}$ **d)** $2^{n\wedge 2}$

# Break

# Complementation

- Let us consider a function f(a, b, c, d,……, 0, 1, +, · ), then the complement of the function is defined as f’(a’, b’, c’,……, z’, 1, 0, ., +).

- i.e. When all the variables are replaced by their compliments, 0→1,1→0, or →and, and → or, then we will be called them compliment of a function.

- We can directly put a bar of the entire Boolean expression to find complement of the function.

- We can also directly deal in minterms and maxterm to write complement function.

# Break

# Duality

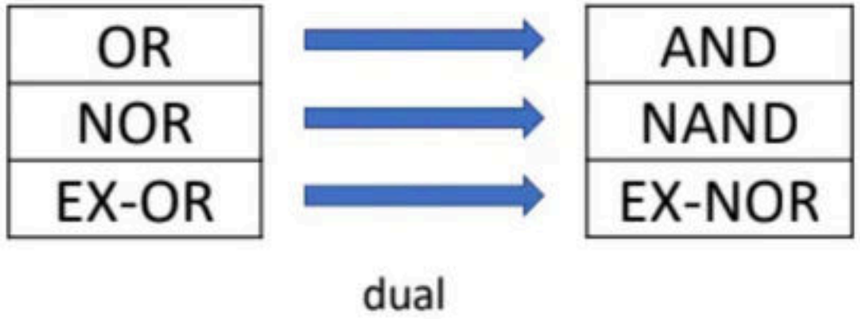
- Let us consider a function f(a, b, c, d,……,z,0,1,+,· ), then the dual of the function is defined as $f^d$(a, b, c,……,z,1,0,·.,+).

- When the nature of variable remains same but 0 → 1, 1 → 0, or → and, and → or, then they are called dual functions.

| a | b | | |
|---|---|---|---|
| L O H | L O H | L O H | L O H |
| L O H | H 1 L | H 1 L | L O H |
| H 1 L | L O H | H 1 L | L O H |
| H 1 L | H 1 L | H 1 L | H 1 L |

- When we take dual of a function, then the functionality of a function remains the same but a positive logic system is transformed to negative logic system.

- If a function works corrects in positive logic system, then it must also work correct in negative logic system as it do not depends on magnitude.



**Q** The dual of a Boolean expression is obtained by interchanging **(NET-DEC-2013)**

**(A)** Boolean sums and Boolean products

**(B)** Boolean sums and Boolean products or interchanging 0's and 1's

**(C)** Boolean sums and Boolean products and interchanging 0's & 1's

**(D)** Interchanging 0's and 1's

**Q** The dual of the switching function x + yz is: **(NET-DEC-2007) (NET-DEC-2008)**

**(A)** x + yz **(B)** x' + y'z' **(C)** x (y + z) **(D)** x' (y' + z')

# Break

# Neutral Function

- A function f is said to be neutral if, it has equal number of minterms and maxterms.

# Self-Dual

- A function f is said to be self-dual if both the functions and its dual are same.
- $f = f^D$
- F (a, b, c) = ab + bc + ca

**Q** which of the following functions are self-dual?

$f(a, b, c) = \sum_m (0, 3)$

$F(a, b, c) = \sum_m (0, 1, 6, 7)$

$F(a, b, c) = \sum_m (0, 1, 2, 4)$

$F(a, b, c) = \sum_m (3, 5, 6, 7)$

**Q** how to check weather a function is self-dual or not?

1) check whether function is neutral or not i.e. (minterm = maxterm)

2) A self-dual function does not have any mutually exclusive terms. So in every pair of mutual exclusion term, we can pick only 1 minterm.

3) for n variable functions total $2^n$ minterms are possible, so we will have $2^{n-1}$ pair of mutually exclusive minterms, in every pair of mutually exclusive minterms we have two choice so, For n variable function total number of $2^{(2^{(n-1)})}$ self-dual functions are possible.

0←→7
1←→6
2←→5
3←→4
e.g. of Mutual exclusive min terms

**Q** The dual of a Boolean function F ($X_1$, $X_2$, ..., $X_n$, +, *, '), written as $F^D$, is the same expression as that of F with + and * swapped. F is said to be self-dual if $F = F^D$. The number of self-dual functions with n Boolean variables is. **(GATE-2014) (1 Marks)**

**a)** $2^n$ **b)** $2^{(n-1)}$ **c)** $2^{(2014\text{^}n)}$ **d)** $2^{(2\text{^}(n-1))}$

**Q** How many Boolean functions of degree n are self-dual? **(NET-JUNE-2014)**

**(A)** $2^n$ **(B)** $(2)^{2n}$ **(C)** 2 n^2 **(D)** (2) ^ ($2^{n-1}$)

# Break

# Orthogonal

- A function f is said to be orthogonal if the compliment and dual of the function are same.

- $f' = f^d$

- $f(a, b, c) = a'b'c' + abc + a'bc' + ab'c$

- A function can never be both orthogonal and self-dual because it will imply the function is same to its compliment, which is never possible.

**Q** how to check weather a function is orthogonal or not?

1) check whether function is neutral or not i.e. (minterm = maxterm)

2) we can choose any pair of mutual exclusive terms, but the minterms and its complement, both must be present.

3) If there is a function f of n-variables, so we will have $2^{n-1}$ pair of mutually exclusive minterms, out of which we have to select exactly half of the pairs ($2^{n-2}$), then $(2^{n-1})C(2^{n-2})$ different orthogonal functions are possible.

0←→7
1←→6
2←→5
3←→4

e.g. of Mutual exclusive min terms